第 11 章 密码学哈希函数

0. 学习目标总览

教材给出本章六个学习目标：

1. 总结密码学哈希函数的应用。
2. 解释使用哈希函数认证消息时要保证其安全的原因。
3. 理解抗原像攻击、抗第二原像攻击和抗强碰撞攻击的区别。
4. 简介密码学哈希函数的基本结构。
5. 描述如何使用分组密码链接来构建哈希函数。
6. 理解 SHA-512 的运算过程。

哈希函数 H 将变长数据分组 M 映射为定长哈希值 $h = H(M)$。密码学哈希函数要求单向性（给定哈希值找原像不可行）和抗碰撞性（找到两个不同消息产生相同哈希值不可行）。因此，哈希函数是判断数据是否被篡改的核心工具。

```
密码学哈希函数
  |
  ├── 应用
  |     ├── 消息认证（配合对称密钥或秘密值）
  |     ├── 数字签名（对哈希值签名）
  |     `-- 其他：口令、入侵检测、PRNG等
  |
  ├── 安全要求
  |     ├── 抗原像攻击（单向性）
  |     ├── 抗第二原像攻击（抗弱碰撞）
  |     `-- 抗碰撞攻击（抗强碰撞）
  |
  ├── 基本结构：迭代哈希函数
  |     ├── 填充 → 分组 M_i
  |     ├── 压缩函数 f(CV_{i-1}, M_i) → CV_i
  |     `-- 分组密码链接构造
  |
  `-- SHA 系列
        ├── SHA-512：1024位分组，80轮
        `-- SHA-3：海绵结构，Keccak-f[1600]
```

1. 学习目标 1：密码学哈希函数的应用

1.1 消息认证

消息认证是验证消息完整性的机制或服务，确保数据在传输或存储过程中未被修改、插入、删除或重放，并验证发送方声称的身份。基本使用方式为：发送方计算 (M) 的哈希值，一起发送；接收方重算哈希值并比对。

教材给出四种哈希码提供消息认证的方案：

方案	方式	特性
a	$E(K, M \parallel H(M))$	提供保密性和认证
b	$M \parallel E(K, H(M))$	仅认证，降低加密开销
c	$M \parallel H(M \parallel S)$	仅认证，使用共享秘密 S，不加密
d	$E(K, M \parallel H(M \parallel S))$	提供保密性 + 认证

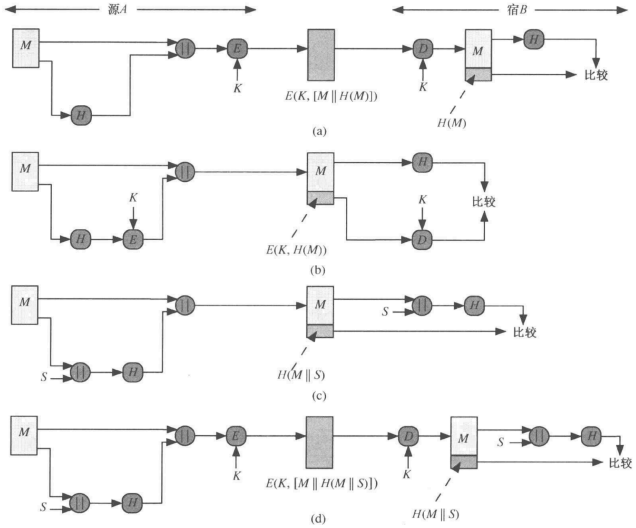


图 11.3　使用哈希函数认证消息的简化示例

图 11.3 消息认证的基本方案

1.2 数字签名

数字签名中，哈希值用发送方的私钥加密。因为只有发送方拥有私钥，验证方可使用发送方的公钥解密并验证哈希值，从而确认签名的真实性和消息的完整性。实际上数字签名是对哈希值的加密签名，而不是对整个消息。

1.3 其他应用

- **生成单向口令文件**：操作系统存储口令的哈希值而非明文，即使口令文件被读取也不能恢复原始口令。
- **入侵检测和病毒检测**：为每个文件存 $H(F)$ 并安全保存；周期重算并比对。
- **构建伪随机函数（PRF）或伪随机数生成器（PRNG）**。

2. 学习目标 2 & 3：哈希函数的安全要求

2.1 七个安全要求

表 11.1 列出了密码学哈希函数 H 的安全要求：

要求	描述
输入长度可变	H 适用于任意长度的数据分组。
输出长度固定	H 生成定长输出。
效率	对任意 x，计算 $H(x)$ 比较容易。
抗原像攻击（单向性）	对任意哈希值 h，找到满足 $H(y) = h$ 的 y 在计算上不可行。
抗第二原像攻击（抗弱碰撞…	对任何 x，找到 $y \neq x$ 且 $H(y) = H(x)$ 的 y 在计算上不可行。
抗碰撞攻击（抗强碰撞）	找到任意 $y \neq x$ 且 $H(y) = H(x)$ 的点对 (x, y) 在计算上不可行。
伪随机性	H 的输出满足伪随机性测试标准。

其中抗原像和抗第二原像是实际消息认证必需的；抗强碰撞对数字签名尤其重要，否则敌手可准备两版含义截然不同却有同一哈希值的消息，签署一版后以另一版抵赖。

抗原像 & 抗第二原像 & 抗碰撞

抗原像攻击、抗第二原像攻击与抗碰撞攻击是哈希函数安全性的三个递进层次，但其递进方向与你所述的恰好相反。三者按**攻击难度从低到高**排列为：**碰撞攻击 < 第二原像攻击 < 原像攻击**。

- **碰撞攻击（最易）**：攻击者只需找到任意一对 (x, y)，$x \neq y$，使得 $H(x) = H(y)$。生日攻击（附录 E）将搜索复杂度降至 $O(2^{n/2})$。这是三类攻击中最弱的一种防御目标。
- **第二原像攻击（较难）**：攻击者给定一个特定的 x，需要找到另一个 $y \neq x$ 使得 $H(y) = H(x)$。其复杂度为 $O(2^n)$，因为攻击者必须针对该固定 x 的哈希值进行搜索，无法利用生日悖论。
- **原像攻击（最难）**：攻击者给定一个哈希值 h，需要找到任意 x 使得 $H(x) = h$。复杂度同为 $O(2^n)$，但通常因目标值 h 是事先给定的，搜索空间的结构约束比第二原像攻击更强。

三者存在严格的能力包含关系：**能够实施原像攻击的攻击者，必然能够实施第二原像攻击**——只需将给定的 x 的哈希值 $H(x)$ 作为原像攻击的输入，返回的 y（若等于 x 则重试）即为所需。**能够实施第二原像攻击的攻击者，必然能够实施碰撞攻击**——只需随意选取一个 x_0 作为输入，得到的 $y_0 \neq x_0$ 即构成一对有效碰撞。

因此，哈希函数的安全强度应表述为：**抗碰撞攻击** $\Rightarrow$ **抗第二原像攻击** $\Rightarrow$ **抗原像攻击**。即：若一个哈希函数能够抵抗碰撞攻击（即攻击者无法找到任意碰撞），则它必然能抵抗第二原像攻击和原像攻击。所谓“攻破抗第二原像”意味着攻击者能够针对任意给定的 x 找到 y，这本身就蕴涵了他能够通过主动选择 x 来构造任意碰撞——因此，**破坏抗第二原像攻击等价于（在计算意义上）能够找到任意碰撞**，而非“不意味着”。

最终结论：**抗碰撞是哈希函数最高级别的安全要求**，而非最弱的一环。一个能抵抗碰撞的哈希函数，自然提供了原像和第二原像的安全性；反之，只依赖单向性（抗原像）的函数，其碰撞安全性可能完全不存在。

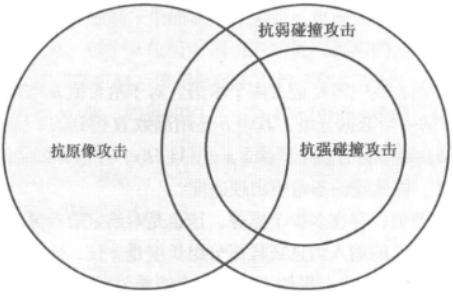


图 11.6　哈希函数的几条性质之间的关系

图 11.6 哈希函数的几条性质之间的关系

一个函数如果是抗强碰撞的，那么它同时也是抗弱碰撞的，反之不一定成立。一个函数可以是抗强碰撞的，但不一定是抗原像攻击的，反之也成立。一个函数可以是抗弱碰撞的，但不一定是抗原像攻击的，反之也成立。

如下表给出了不同数据完整性应用中哈希函数的安全要求。

	抗原像攻击	抗弱碰撞攻击	抗强碰撞攻击
哈希+数字签名	是	是	是*
入侵检测和病毒检测		是	
哈希+对称加密			
单向口令文件	是		
MAC	是	是	是*

- 要求低手能够实现选择消息攻击。

2.2 原像和碰撞的基本数量关系

设 H 的哈希码长度为 n 位，输入消息长度为 b 位（$b > n$）。可能的消息总数为 2^b，哈希值总数为 2^n。平均每个哈希值对应 2^{b-n} 个原像。如果 H 满足均匀分布，每个哈希值就对应近 2^{b-n} 个原像。

2.3 穷举攻击（生日攻击）

对抗原像和第二原像攻击，穷举攻击需约 2^n 次尝试。对抗碰撞攻击，由于"生日悖论"，仅需约 $2^{n/2}$ 次尝试即可找到碰撞——这与"随机选取 k 个人时，只需约 $\sqrt{N}$ 个人就有 50% 概率两人生日相同"同理。

因此，n 位哈希码的抗碰撞强度只有 $n/2$ 位。这就是 SHA-1（160 位）需 2^{80} 次操作才能碰撞，而现今标准要求 256 位以上（抗碰撞强度 128 位）的原因。

3. 学习目标 4 & 5：基本结构与分组密码构造

3.1 迭代哈希函数的一般结构（MD 结构）

大多数哈希函数采用迭代结构：将输入消息分为固定长度的分组，后一分组的输入依赖于前一分组的输出。

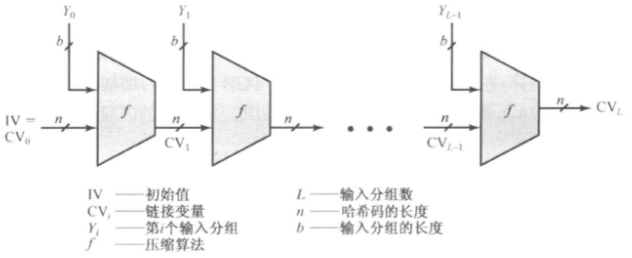


图 11.8　安全哈希函数的总体结构

图 11.8 迭代哈希函数的一般结构

$$CV_0 = IV, \qquad CV_i = f(CV_{i-1}, Y_{i-1}), \quad i = 1, \ldots, L$$

$$H(M) = CV_L$$

其中 IV 是初始值，CV 是链接变量，Y_i 是第 i 个消息分组，f 是压缩函数。若 f 抗碰撞，则整个哈希函数也抗碰撞。

任何哈希函数都存在碰撞，因此我们所要求的只是找到碰撞在计算上是不可行的。

3.2 用分组密码构造哈希函数

教材介绍两类分组密码链接构造：Rabin 方案中，$CV_i = E(M_i, CV_{i-1})$——消息分组作为密钥对前一链接变量加密。更安全的结构是 Davies-Meyer 方案，在密码之中加入了额外的反馈：$CV_i = E(M_i, CV_{i-1}) \oplus CV_{i-1}$。其抗碰撞性依赖于差分密码分析的困难性。

4. 学习目标 6：SHA-512 的运算过程

4.1 SHA 系列概述

算法	消息长度	分组长度	字长度	摘要长度
SHA-1	$< 2^{64}$	512	32	160
SHA-256	$< 2^{64}$	512	32	256
SHA-512	$< 2^{128}$	1024	64	512
SHA-512/256	$< 2^{128}$	1024	64	256

SHA-1 已在 2017 年被实际碰撞攻击攻破，目前被 SHA-2 取代。SHA-512 是 SHA-2 中摘要最长的版本。

4.2 SHA-512 的处理流程

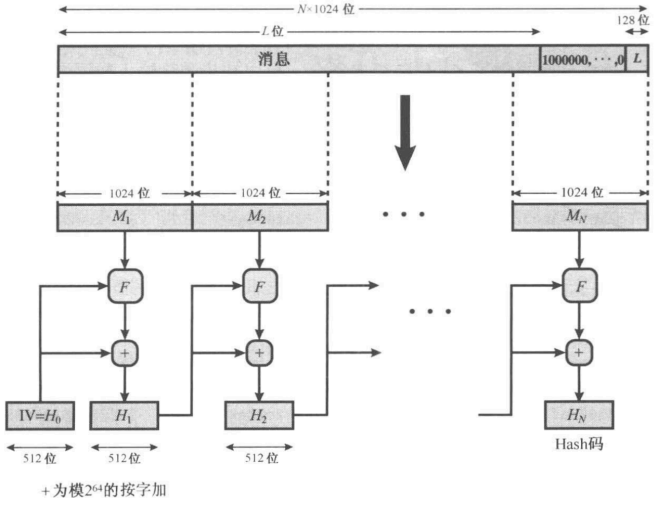


图 11.9　使用 SHA-512 生成消息摘要

图 11.9 SHA-512 从消息到摘要的过程

1. **附加填充位。** 填充消息使长度 $\equiv 896 \pmod{1024}$；首位填 1，其余填 0。无论消息原长是否满足此条件都填充，因此填充数的范围是 $1 \sim 1024$。
2. **附加长度。** 在末尾追加 128 位无符号整数，表示填充前原始消息的总位数。最终消息长度是 1024 的整数倍。
3. **初始化哈希缓冲区。** 512 位缓冲区存于 8 个 64 位寄存器 (a, b, c, d, e, f, g, h)，初始值取自前 8 个素数的平方根小数部分的前 64 位。
4. **以 1024 位分组处理消息。** 每个分组经 80 轮迭代压缩函数处理。每轮都把512位缓冲值abcdefgh作为输入，并更新缓冲区的内容。在第一轮，缓冲区中的值是中间值 H_{i-1}。每轮（如第t轮）使用一个64位值 W_t.，该值由当前被处理的1024位消息分组M_i 导出，导出算法是下面将要讨论的消息扩展算法。每轮还使用附加常数 K_t，其中 $0 \leqslant t \leqslant 79$,以便每轮的运算不同。这些常数的获得方法如下：取前80个素数的立方根，并取小数部分的前 64位。这

些常数提供了64位随机串集合，可以初步消除输入数据中的统计规律。

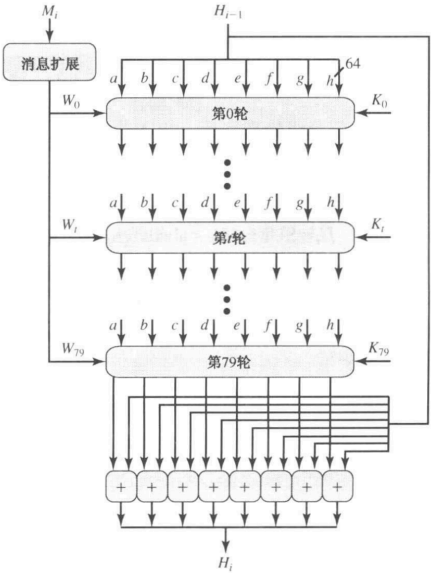


图 11.10　SHA-512 对单个 1024 位分组的处理

5. **输出。** 最后一个分组处理完后，输出 512 位消息摘要。

SHA-512 运算小结：

$$
\begin{aligned}
H_0 &= \mathrm{IV} \\
H_i &= \mathrm{SUM}_{64}(H_{i-1}, \mathrm{abcdefgh}_i) \\
\mathrm{MD} &= H_N
\end{aligned}
$$

式中，IV 是步骤 3 中定义的 $abcdefgh$ 缓冲区的初始值；abcdefghi 表示第 i 个消息分组处理的最后一轮的输出；N 表示消息（包括填充和长度字段）中的分组数； SUM_{64} 表示对输人对中的每个字单独进行模 2^{64} 加；MD 表示最后的消息摘要值。

4.3 SHA-512 轮函数

每轮用 64 位的五个操作函数和一个 64 位消息字 W_t、一个 64 位轮常数 K_t 来更新寄存器：

$$
\begin{aligned}
T_1 &= h + \mathrm{Ch}(e,f,g) + \left(\sum\nolimits_1^{512} e\right) + W_t + K_t \\
T_2 &= \left(\sum\nolimits_0^{512} a\right) + \mathrm{Maj}(a,b,c) \\
h &= g, \quad g = f, \quad f = e, \quad e = d + T_1 \\
d &= c, \quad c = b, \quad b = a, \quad a = T_1 + T_2
\end{aligned}
$$

其中 $\mathrm{Ch}(e,f,g) = (e \wedge f) \oplus (\neg e \wedge g)$（选择函数），$\mathrm{Maj}(a,b,c) = (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c)$（多数函数）。

$\left(\sum_0^{512} a\right) = \mathrm{ROTR}^{28}(a) + \mathrm{ROTR}^{34}(a) \oplus \mathrm{ROTR}^{39}(a)$；　$\left(\sum_1^{512} e\right) = \mathrm{ROTR}^{14}(e) +$
$\mathrm{ROTR}^{18}(e) \oplus \mathrm{ROTR}^{41}(e)$；

ROTR(x)表示对 64 位变量 x 循环右移 n 位； W_t 是一个 64 位字，它由当前的 1024 位输入分组导出。

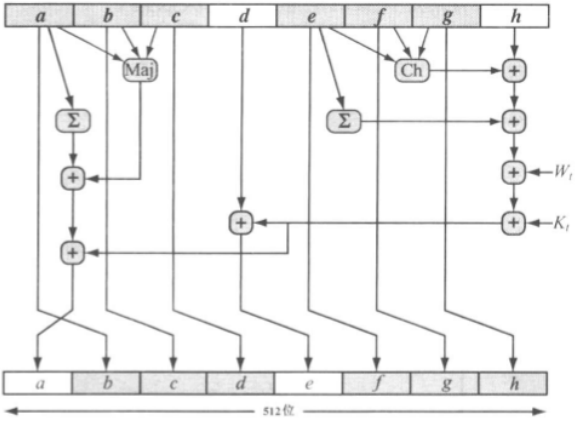


图 11.11　基本的 SHA-512 运算（单轮）

图 11.11 SHA-512 单轮运算

> 观察可知，轮函数具有如下两个特点：
> 1.轮函数输出的8个字中的6个 (b, c, d, f, g, h) 是借助轮转置换实现的，如图11.11中的阴影部分所示。
>
> 2.输出中只有2个字 (a, e) 通过替代操作输出中只有2个字 (a, e) 通过替代操作生成。字e是以输人变量 (d, e, f, g, h) 及轮字 W_t 和常数 K_t 为输入的函数。字 a 是以除 d 外的输入变量及轮字 W_t 和常数 K_t 为输入的函数。

轮常数 K_t： 80 个 64 位常数，取自前 80 个素数的立方根小数部分的前 64 位。

消息调度（扩展）： 16 个 64 位字 $W_0, \ldots, W_{15}$ 直接取自当前 1024 位分组；$t = 16$ 到 79 时按如下方式扩展：

$$W_t = \sigma_1^{512}(W_{t-2}) + W_{t-7} + \sigma_0^{512}(W_{t-15}) + W_{t-16}$$

示模 2^{64} 加。

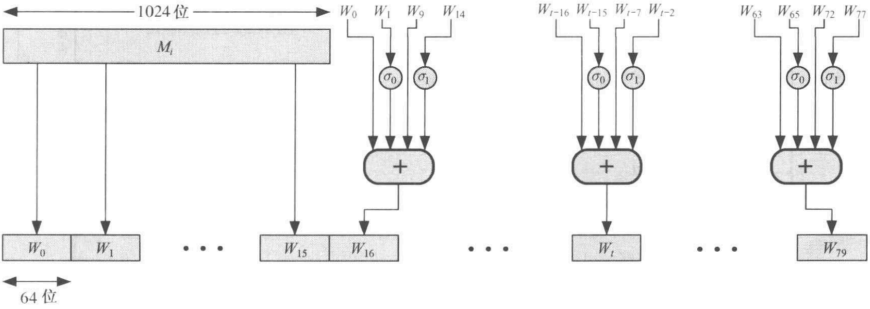


图 11.12　SHA-512 处理单个分组生成的 80 字输入序列

式中： $\sigma_0^{512}(x) = \mathrm{ROTR}^1(x) + \mathrm{ROTR}^8(x) + \mathrm{SHR}^7(x)$ ；
$\sigma_1^{512}(x) = \mathrm{ROTR}^{19}(x) + \mathrm{ROTR}^{61}(x) + \mathrm{SHR}^6(x)\ \mathrm{ROTR}^n(x)$
表示对 64 位变量 x 循环右移 n 位； $\mathrm{SHR}^n(x)$ 表示对 64 位变量 x 右移 n 位，左侧填充 0；+表示模 2^{64} 加。

因此，在前 16 步处理中， W_t 的值等于消息分组中的相应字。对余下的 64 步， W_t 的值由前面的 4 个值推导得出，这 4 个值中的两个要进行算术移位和循环移位运算，于是为被压缩消息分组引入了大量冗余性和相互依赖性，使得找出具有相同压缩函数输出的不同消息的任务更加复杂。图 11.13 总结了 SHA-512 逻辑。

填充后的消息由分组 $M_1, M_2, \cdots, M_N$ 组成。每个消息分组 M_i 由 16 个 64 位字 $M_{i,0}, M_{i,1}, \cdots, M_{i,15}$ 组成。所有加法都是模 2^{64} 加。

$H_{0,0} = 6A09E667F3BCC908$　　$H_{0,4} = 510E527FADE682D1$

$H_{0,1} = BB67AE8584CAA73B$　　$H_{0,5} = 9B05688C2B3E6C1F$

$H_{0,2} = 3C6EF372FE94F82B$　　$H_{0,6} = 1F83D9ABFB41BD6B$

$H_{0,3} = A54FF53A5F1D36F1$　　$H_{0,7} = 5BE0CD19137E2179$

for $i = 1$ **to** N

1. 准备消息调度 W

for $t = 0$ **to** 15

$W_t = M_{i,t}$

for $t = 16$ **to** 79

$W_t = \sigma_1^{512}(W_{t-2}) + W_{t-7} + \sigma_0^{512}(W_{t-15}) + W_{t-16}$

2. 初始化工作变量

$a = H_{i-1,0}$　　$e = H_{i-1,4}$

$b = H_{i-1,1}$　　$f = H_{i-1,5}$

$c = H_{i-1,2}$　　$g = H_{i-1,6}$

$d = H_{i-1,3}$　　$h = H_{i-1,7}$

3. 执行主哈希计算

for $t = 0$ **to** 79

$T_1 = h + \mathrm{Ch}(e, f, g) + (\sum_1^{512} e) + W_t + K_t$

$T_2 = (\sum_0^{512} a) + \mathrm{Maj}(a, b, c)$

$h = g$

$g = f$

$f = e$

$e = d + T_1$

$d = c$

$c = b$

$b = a$

$a = T_1 + T_2$

4. 计算中间哈希值

$H_{i,0} = a + H_{i-1,0}$　　$H_{i,4} = e + H_{i-1,4}$

$H_{i,1} = b + H_{i-1,1}$　　$H_{i,5} = f + H_{i-1,5}$

$H_{i,2} = c + H_{i-1,2}$　　$H_{i,6} = g + H_{i-1,6}$

$H_{i,3} = d + H_{i-1,3}$　　$H_{i,7} = h + H_{i-1,7}$

return $\{H_{N,0} \| H_{N,1} \| H_{N,2} \| H_{N,3} \| H_{N,4} \| H_{N,5} \| H_{N,6} \| H_{N,7}\}$

图 11.13　SHA-512 逻辑

SHA-512 算法性质

哈希码的每一位都是全部输入位的函数。基本函数F多次重复运算后，使得结果充分混淆；也就是说，不可能随机选择具有相同哈希码的两条消息，即使它们具有类似的规律性。除非 SHA-512中存在未公开的缺陷，否则找到两条具有相同摘要的消息的运算次数为 2^{256}

4.4 教材 SHA-512 示例

教材以字符串"abc"为例演示 SHA-512。消息"abc"的 ASCII 二进制长度为 24 位。填充：

- 1 位"1"，随后 871 位"0"（达到 896 位），最后追加 128 位表示长度 24。
- 因此仅有单个 1024 位消息分组。

经过 80 轮运算后，8 个寄存器的输出与初始 IV 相加，得到最终的 512 位摘要。教材演示了一个 1024 位输入的第二个例（448 位"a"后填充），得到的摘要与前例的摘要恰好改变了 253 位——几乎一半，说明 SHA-512 具有良好的雪崩效应。

5. 补充：SHA-3 的海绵结构

5.1 背景与海绵结构定义

SHA-1 与已被攻破的 MD5 和 SHA-0 具有类似结构，存在风险。SHA-2 虽暂时安全，但设计上与 SHA-1 类似。因此 NIST 于 2007 年公开征集 SHA-3，2012 年选定 Keccak，2015 年发布 FIPS 202。

SHA-3 的基础是**海绵结构**，由三个参数定义：

参数	含义
f	内部函数，处理每轮输入分组
r	位速率，即输入分组的位长度
pad	填充算法（FIPS 202 使用多速率填充 pad10*1）

海绵结构对状态变量 s 进行操作，其长度 $b = r + c$，其中 c 是**容量**（决定安全程度）。Keccak 默认：$c = 1024$、$r = 576$，故 $b = 1600$。

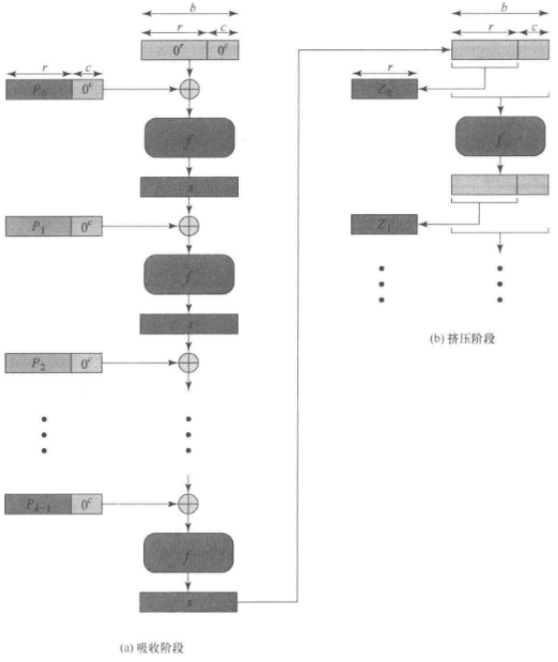


图 11.15　海绵函数的迭代结构

图 11.15 海绵函数的迭代结构

吸收阶段： 每轮将输入分组扩展为 b 位后与 s 异或，再经 f 函数处理，输出作为下一轮的 s。
挤压阶段： 若吸收阶段完成后还需要更多输出位，则不断执行 f 并从 s 的前 r 位取输出。

> **理解**
>
> 海绵结构是SHA-3的核心构造，它对长度为 $b = r + c$ 位的状态变量 s 进行操作，初值全为 0，并在每轮迭代中更新。其中r 称为位速率，决定每轮处理的消息分组长度，r 越大处理速度越快；c 称为容量，决定安全强度，c 越大安全性越高，但速度相应降低，二者存在权衡。
>
> 算法分两个阶段：吸收阶段对每个输入分组填充 0 至 b 位后与 s 异或，经迭代函数 f 更新状态；若期望输出长度 $\ell \le r$，则直接返回 s 的前 r 位并结束；否则进入挤压阶段，先输出 s 的前 r 位作为 Z_0，再重复执行 f 更新状态，逐轮输出 Z_i 的前 r 位并拼接，直至累积长度达到 ℓ。海绵结构将哈希、伪随机数生成等功能统一在同一框架下，灵活性极高。

5.2 Keccak-f 迭代函数

SHA-3 使用 Keccak-f[1600]，其状态可表示为 $5 \times 5 \times 64$ 的三维数组 $a[x][y][z]$（x, y 为通道坐标，z 为位坐标），共 24 轮，每轮包含五个阶跃函数：

阶跃	名称	作用
θ	扩散	将每位的奇偶性扩散到相邻列。
ρ	通道内旋转	每个通道内位循环移位不同常数，提供通道内扩散。
π	通道间置换	在 5×5 矩阵内移动各通道位置。
χ	非线性	唯一非线性运算：$a[x] \leftarrow a[x] \oplus ((\neg a[x+1]) \wedge a[x+2])$。
ι	轮常数	将每轮不同常数 XOR 到第一个通道，打破对称性。

本章关键术语整理

术语	教材含义
哈希函数	将变长数据映射为定长哈希值的函数。
消息摘要/哈希码/哈希值	哈希函数的输出，作为消息的数字指纹。
消息认证码（MAC）	使用秘密密钥的哈希函数，提供消息完整性和认证。
抗原像攻击	给定哈希值 h 找不到原像 y 使 $H(y) = h$ 的性质。
抗碰撞攻击	找不到 $x \neq y$ 且 $H(x) = H(y)$ 的性质。
生日攻击	利用生日悖论，仅需 $2^{n/2}$ 次尝试即可找到 n 位哈希的碰撞。
迭代哈希函数	将消息分组后依次处理的哈希结构。
压缩函数	将当前链接变量和消息分组映射为新链接变量的函数。
SHA-512	SHA-2 中输出 512 位、使用 1024 位分组的哈希算法。
SHA-3 / Keccak	基于海绵结构的新一代哈希标准。
海绵结构	由吸收和挤压两阶段组成的迭代结构。

学习目标自检

1. 能列出哈希函数在消息认证、数字签名、口令存储、入侵检测和 PRNG 中的用途。
2. 能说明在使用哈希码进行消息认证时，若哈希值不被保护（如中间人攻击），敌手可如何绕过检测。
3. 能区分抗原像（单向性）、抗第二原像（弱碰撞）和抗强碰撞三种安全要求及其针对的攻击场景。
4. 能解释生日攻击的原理，并说明 n 位哈希码的抗碰撞强度仅约为 $n/2$ 位。
5. 能画出迭代哈希函数的基本结构 $CV_i = f(CV_{i-1}, Y_{i-1})$，并说明压缩函数抗碰撞是整体抗碰撞的充分条件。
6. 能说明用分组密码构建哈希函数时，Davies-Meyer 方案与单纯加密方案的区别。
7. 能写出 SHA-512 的填充规则（长度模 1024 余 896 + 128 位长度），以及 80 轮中 T_1、T_2 和寄存器更新关系。
8. 能说明海绵结构两个阶段的工作原理，以及位速率 r 和容量 c 对速度和安全的影响。
9. 能列出 Keccak-f 五个阶跃函数（θ、ρ、π、χ、ι）各自的作用。